

UNIVERSIDAD PANAMERICANA

Estudiante : Andoreni Claudio Flores Alcocer
ID : 0246717
Materia : Cómputo Distribuido
Profesor : Carlos Pérez Leguizamo
Proyecto Primer Parcial : Calculadora Distribuida

1. Introducción y objetivo

Este documento describe el diseño técnico y la construcción de una aplicación distribuida que implementa las cuatro operaciones básicas (suma, resta, multiplicación y división) mediante una arquitectura cliente/servidor de tres capas, utilizando sockets TCP como mecanismo de comunicación. El sistema está compuesto por tres tipos de componentes independientes : Cliente, Middleware y Servidor. Se comunican exclusivamente a través de mensajes intercambiados por sockets, sin acoplamiento directo entre clientes y servidores.

El desarrollo se realizó en Python 3, utilizando únicamente librerías de la biblioteca estándar : **socket** para la comunicación en red, **threading** para el manejo concurrente de múltiples conexiones, **sqlite3** para la persistencia de información y **tkinter** para la interfaz gráfica del cliente.

2. Arquitectura general

La solución sigue una arquitectura de tres capas claramente separadas :

Capa de presentación (Cliente pesado): aplicación de escritorio con interfaz gráfica, responsable únicamente de capturar la operación solicitada por el usuario, enviarla y mostrar los resultados recibidos.

Capa de comunicación (Middleware): módulo intermedio que recibe todas las conexiones, tanto de clientes como de servidores, identifica el tipo de cada conexión y se encarga exclusivamente de enrutar y distribuir mensajes entre ambos lados.

Capa de negocio (Servidor): componente responsable de resolver las operaciones aritméticas solicitadas y de persistir sus propios resultados.

Estas tres capas se ejecutan como procesos independientes (pueden estar en la misma máquina o en máquinas distintas de la misma red) y se comunican exclusivamente mediante sockets TCP, nunca de forma directa entre cliente y servidor. Esto asegura que el middleware sea el único punto de contacto entre ambos extremos y que las capas puedan escalar de forma independiente; es posible conectar múltiples clientes y múltiples servidores de forma simultánea sin modificar el código de ninguno de los componentes.

Cliente 1, Cliente 2, ..., Cliente N ↔ (sockets TCP) ↔ Middleware ↔ (sockets TCP) ↔ Servidor 1, Servidor 2, ..., Servidor N

3. Componentes del sistema

3.1 Cliente pesado

Aplicación de escritorio construida con Tkinter (no es una interfaz web ni un navegador). Su responsabilidad se limita a capturar del usuarios los dos operandos y la operación a realizar; construir una solicitud y enviarla al middleware; recibir en segundo plano los resultados que el middleware distribuye y mostrarlos en una tabla y persistir localmente tanto las solicitudes que este cliente generó como todos los resultados que recibió.

El cliente no contiene lógica de negocio, en ningún momento calcula el resultado de la operación, únicamente la solicita y muestra lo que el servidor respondió. La recepción de mensajes se ejecuta en un hilo separado del hilo principal de la interfaz gráfica, para no bloquear la ventana mientras se espera la llegada de resultados, la comunicación entre ese hilo de red y la interfaz se realiza mediante una cola (queue) que la interfaz consulta periódicamente, evitando así actualizar widgets de Tkinter desde un hilo distinto al principal.

3.2 Middleware

Servidor de sockets que escucha en un puerto TCP y acepta conexiones tanto de clientes como de servidores por el mismo puerto. Cada conexión, al establecerse, debe enviar como primer mensaje un mensaje de registro (REGISTER) indicando su rol (“client” o “server”) y un identificador propio, con esa información el middleware clasifica la conexión y la agrega a una de dos listas internas (clientes conectados y servidores conectados).

A partir de ese momento, el middleware ejecuta dos reglas de enrutamiento simples, toda solicitud (REQUEST) proveniente de un cliente se reenvía a todos los servidores actualmente conectados. El middleware no interpreta ni calcula el contenido de las operaciones, ni las persiste, su única responsabilidad es la comunicación y distribución de mensajes, manteniendo así una separación clara de responsabilidades respecto a la otras dos capas.

Cada conexión es atendida en un hilo dedicado, lo que permite el middleware maneja múltiples servidores de forma simultánea. El acceso a las listas compartidas de conexiones está protegido con un candado (lock) para evitar condiciones de carrera cuando distintos hilos agregan, eliminan o recorren esas listas al mismo tiempo.

3.3 Servidor

Proceso que se conecta al middleware como cliente de socket, se registra con el rol “server” y queda a la espera de solicitudes reenviadas por el middleware. Al recibir una solicitud, el servidor ejecuta la operación aritmética correspondiente (suma, resta, multiplicación o división), maneja el caso de división entre cero como un error controlado (sin detener el proceso), persiste el resultado exitoso o con error en su propia base de datos y finalmente envía el resultado de vuelta al middleware para que este lo distribuya a los clientes.

El servidor concentra toda la lógica de negocio del sistema y la persistencia de sus propios resultados, no tiene conocimiento de los clientes ni de otros servidores y toda su comunicación para exclusivamente por el middleware.

4. Protocolo de comunicación

La comunicación entre los tres componentes se realiza mediante mensajes en formato JSON, enviados sobre sockets TCP. Dado que TCP es un protocolo orientado a flujo de bytes (no preserva los límites de cada mensaje), se definió un formato de “framing” simple, cada mensaje se realiza como una línea de texto JSON terminada con un salto de línea **(\n)**. El receptor va acumulando los bytes recibidos hasta encontrar un salto de línea, momento en el que interpreta ese fragmento como un mensaje JSON completo. Este mecanismo permite enviar múltiples mensajes seguidos por el mismo socket sin ambigüedad sobre dónde termina uno y empieza el siguiente.

Se definieron los siguientes tipos de mensaje :

REGISTER (cliente/servidor → middleware): indica el rol de la conexión ("client" o "server") y un identificador propio. Es obligatoriamente el primer mensaje de cada conexión.

REQUEST (cliente → middleware → todos los servidores): representa una solicitud de operación. Contiene un identificador único de solicitud (request_id, generado como UUID), el identificador del cliente que la originó, la operación (+, -, *, /), los dos operandos y una marca de tiempo.

RESULT (servidor → middleware → todos los clientes): representa el resultado producido por un servidor para una solicitud específica. Contiene el mismo request_id de la solicitud original (para poder correlacionarlo), el identificador del cliente que la originó, el identificador del servidor que lo procesó, la operación, los operandos, el resultado (o nulo si hubo error), un estado ("ok" o "error"), el mensaje de error si aplica, y una marca de tiempo.

El uso de un identificador único por solicitud (request_id) es la decisión de diseño que permite, del lado del cliente, correlacionar cada resultado recibido con la

operación que lo originó, incluso cuando —como se explica en la siguiente sección— un mismo cliente puede recibir varios resultados para una misma solicitud (uno por cada servidor conectado), o resultados de solicitudes hechas por otros clientes.

5. Flujo de comunicacióN

El sistema implementa el siguiente flujo, tal como lo requieren las instrucciones del proyecto :

1. Un cliente genera una solicitud de operación aritmética a partir de lo capturado en su interfaz gráfica.
2. El cliente envía la solicitud al middleware a través de su socket.
3. El middleware identifica que el mensaje proviene de una conexión registrada como cliente y reenvía la solicitud, sin modificarla, a todos los servidores actualmente conectados.
4. Cada servidor conectado procesa la operación solicitada de forma independiente.
5. Cada servidor envía su resultado (correcto o de error) de vuelta al middleware.
6. El middleware identifica que el mensaje proviene de una conexión registrada como servidor y reenvía ese resultado a todos los clientes actualmente conectados, no únicamente al cliente que originó la solicitud.
7. Cada cliente recibe el resultado, lo muestra en su interfaz y lo registra (persiste) en su base de datos local, junto con la solicitud que él mismo generó.

Una consecuencia directa de este es que, si hay dos servidores conectados, cada solicitud produce dos resultados independientes (uno por servidor) y ambos resultados llegan a todos los clientes conectados, incluyendo clientes distintos al que originó la solicitud. Esto se decidió así porque las instrucciones del proyecto especifican explícitamente que el middleware “reenvía la solicitud a todos los servidores conectados” y luego “reenvía los resultados recibidos a todos los clientes conectados”, sin restringir la distribución únicamente al cliente originador. en la interfaz del cliente, cada fila de resultado muestra de qué cliente se originó la solicitud y qué servidor la procesó, de modo que este comportamiento de difusión (broadcast) sea visible y verificable durante la demostración.

6. Decisiones de diseño

Separación estricta de responsabilidades entre capas: el cliente no calcula operaciones, el middleware no calcula ni persiste, y el servidor no gestiona conexiones de clientes. Esta separación se refleja directamente en la organización del código: cada capa está implementada en un archivo/módulo independiente y solo se comunica con las demás a través del protocolo de mensajes descrito, nunca mediante llamadas directas a funciones de otro componente.

Concurrencia basada en hilos: tanto el middleware como el cliente utilizan hilos para no bloquear su operación principal mientras esperan datos de red. El middleware atiende cada conexión entrante en un hilo dedicado, lo que le permite escalar a múltiples clientes y servidores simultáneos sin que una conexión bloquee a las demás. El cliente separa la recepción de mensajes de red (hilo secundario) de la interfaz gráfica (hilo principal), sincronizándolos mediante una cola thread-safe.

Identificación de conexiones mediante mensaje de registro explícito: en lugar de usar el número de puerto o alguna otra heurística, se optó por que cada conexión declare explícitamente su rol al establecerse (REGISTER). Esto hace evidente y auditable, en los logs del middleware, cuáles conexiones son clientes y cuáles son servidores, y evita ambigüedad si en el futuro se agregan más tipos de nodos.

Manejo explícito de errores de negocio: la división entre cero no se trata como una excepción no controlada que tumbe al servidor, sino como un resultado válido del protocolo con estado "error" y un mensaje descriptivo, que se persiste y se distribuye igual que un resultado exitoso. Esto permite demostrar en la ejecución real cómo el sistema maneja casos inválidos sin caerse.

Formato de mensajes JSON con framing por líneas: se eligió JSON por ser un formato ligero, legible y con soporte nativo en la biblioteca estándar de Python (sin dependencias externas), y el framing por saltos de línea porque es la forma más simple de resolver el problema de "dónde termina un mensaje" sobre un socket TCP sin tener que implementar un protocolo binario con longitudes de encabezado.

Persistencia desacoplada por nodo: cada servidor y cada cliente maneja su propia base de datos SQLite local e independiente, en lugar de una base de datos centralizada compartida. Esta decisión refuerza la independencia de cada componente (ninguno depende de que otro esté disponible para poder guardar su propia información) y facilita mostrar, durante la demostración, la persistencia individual de cada nodo.

7. Persistencia de información

Se utilizó SQLite como motor de persistencia, por ser una base de datos embebida (no requiere instalar ni levantar un servidor de base de datos aparte), soportada de forma nativa por la biblioteca estándar de Python(**sqlite3**) y fácilmente inspeccionable con herramientas estándar para efectos de la demostración.

Persistencia en el servidor: cada servidor crea y mantiene su propio archivo de base de datos (uno por identificador de servidor), con una tabla **results** que almacena, por cada operación procesada: el identificador de la solicitud, el identificador del cliente que la originó, el identificador de este servidor, la operación, los dos operandos, el resultado (nulo si hubo error), el estado ("ok" o "error"), el mensaje de error si aplica, y la marca de tiempo del procesamiento. Con esto se cumple el requisito de que "el servidor deberá persistir los resultados de las operaciones que procese".

Persistencia en el cliente: cada cliente crea y mantiene su propio archivo de base de datos (uno por identificador de cliente), con dos tablas. La tabla **requests** almacena las operaciones que ese cliente solicitó (identificador de solicitud, operación, operandos y marca de tiempo). La tabla **results** almacena todos los resultados que ese cliente recibió del middleware de cualquier servidor, y correspondientes tanto a solicitudes propias como a solicitudes de otros clientes, dado el comportamiento de difusión descrito en la sección 5, incluyendo el servidor que lo procesó y el estado del resultado. Con esto se cumple el requisito de que "cada cliente deberá persistir tanto las operaciones que solicite como los resultados que reciba del servidor".

El acceso a cada base de datos está protegido con un candado (lock) a nivel de proceso, ya que las escrituras pueden ocurrir desde distintos hilos (por ejemplo, el hilo de red del cliente escribiendo resultados mientras el hilo principal de la interfaz gráfica registra una nueva solicitud).

8. Concurrencia y escalabilidad

El diseño soporta múltiples clientes y múltiples servidores conectados simultáneamente sin cambios de código, ya que el middleware no asume un número fijo de conexiones: mantiene dos colecciones dinámicas (clientes y servidores conectados) que se actualizan al registrarse o desconectarse cada nodo, y difunde los mensajes recorriendo esas colecciones en el momento del envío. Esto se verificó de forma concreta durante las pruebas, ejecutando dos servidores y dos clientes de manera simultánea (ver siguiente sección), cumpliendo el mínimo de evidencia solicitado en las instrucciones del proyecto.

9. Pruebas realizadas

Además de la ejecución manual de los cinco procesos (middleware, dos servidores, dos clientes) descrita en el video de demostración, se construyó una prueba automatizada de extremo a extremo que:

Levanta el middleware y dos servidores como procesos independientes.

Simula dos clientes mediante conexiones de socket directas (usando el mismo protocolo que usa la interfaz gráfica real), para validar la lógica de comunicación sin depender de una interfaz visual.

Envía las cuatro operaciones básicas, incluyendo un caso de división entre cero, y verifica que: cada solicitud es respondida por ambos servidores; ambos clientes reciben todos los resultados (comportamiento de difusión); el caso de división entre cero se recibe con estado de error, en lugar de bloquear el sistema; y el número de registros persistidos en la base de datos de cada servidor coincide con el número de operaciones enviadas.

Esta prueba se ejecutó exitosamente antes de la entrega, confirmando el correcto funcionamiento del flujo obligatorio de principio a fin.

10. Conclusiones

El sistema desarrollado cumple con los requerimientos funcionales y arquitectónicos establecidos en las instrucciones del proyecto: implementa las cuatro operaciones aritméticas básicas, utiliza una arquitectura cliente/servidor de tres capas comunicadas exclusivamente por sockets TCP, distingue explícitamente las conexiones de clientes y de servidores en el middleware, soporta múltiples clientes y múltiples servidores de forma simultánea, sigue el flujo de comunicación obligatorio (incluyendo la difusión de solicitudes hacia todos los servidores y de resultados hacia todos los clientes), persiste la información tanto en el servidor como en cada cliente, y mantiene una separación clara de responsabilidades entre las tres capas del sistema.